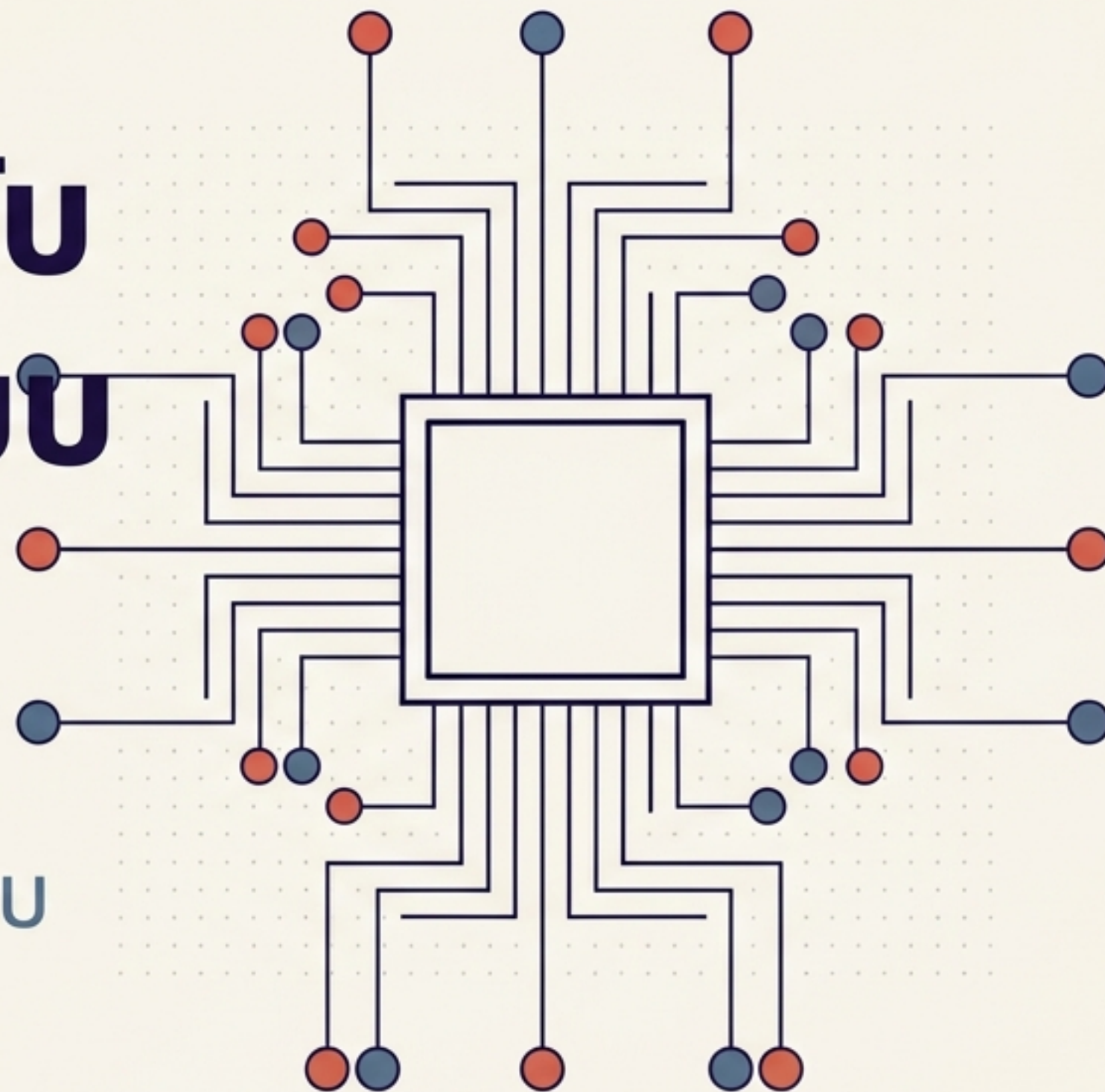


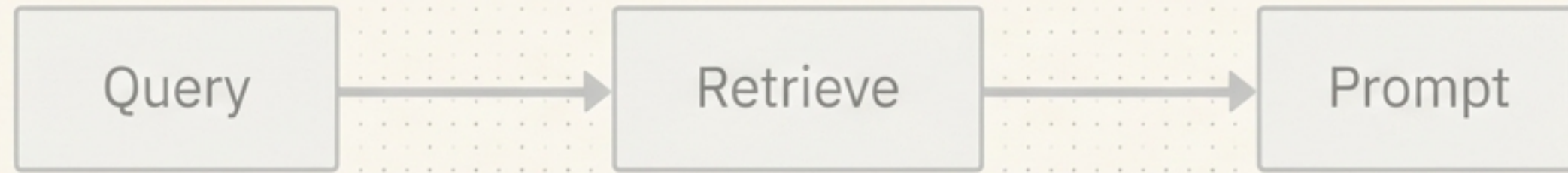
เราพร้อมหรือยังสำหรับ ระบบหน่วยความจำแบบ Agent-Native?

ก้าวข้ามข้อจำกัดของ RAG
สู่สถาปัตยกรรมการจัดการข้อมูลเต็มรูปแบบ

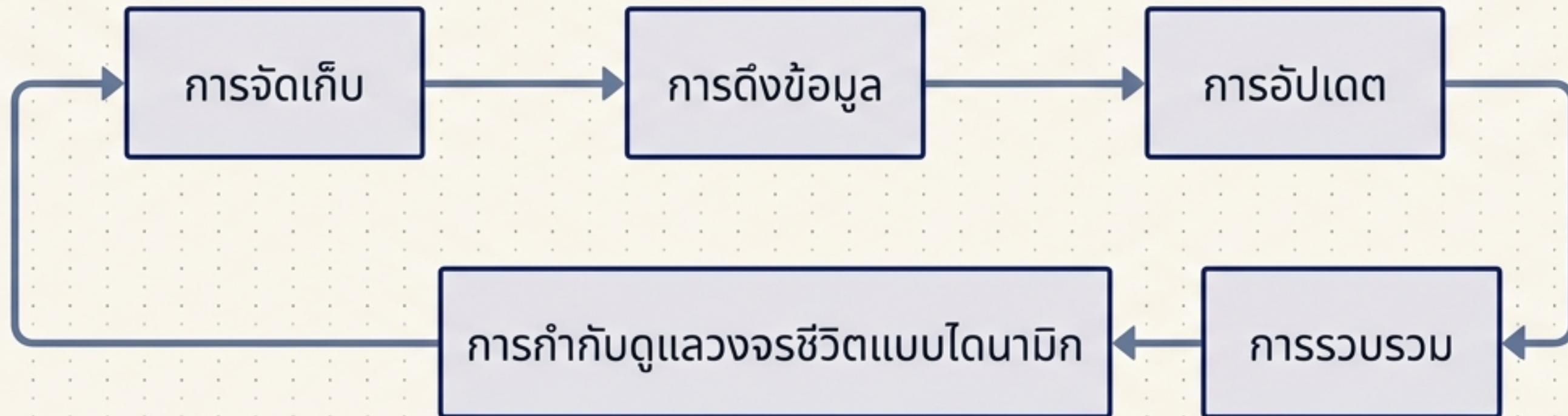


สร้างจากผลงานวิจัย: [2606.24775] Are We Ready For An Agent-Native Memory System?

วิวัฒนาการจาก Simple RAG สู่ Dynamic Data Management

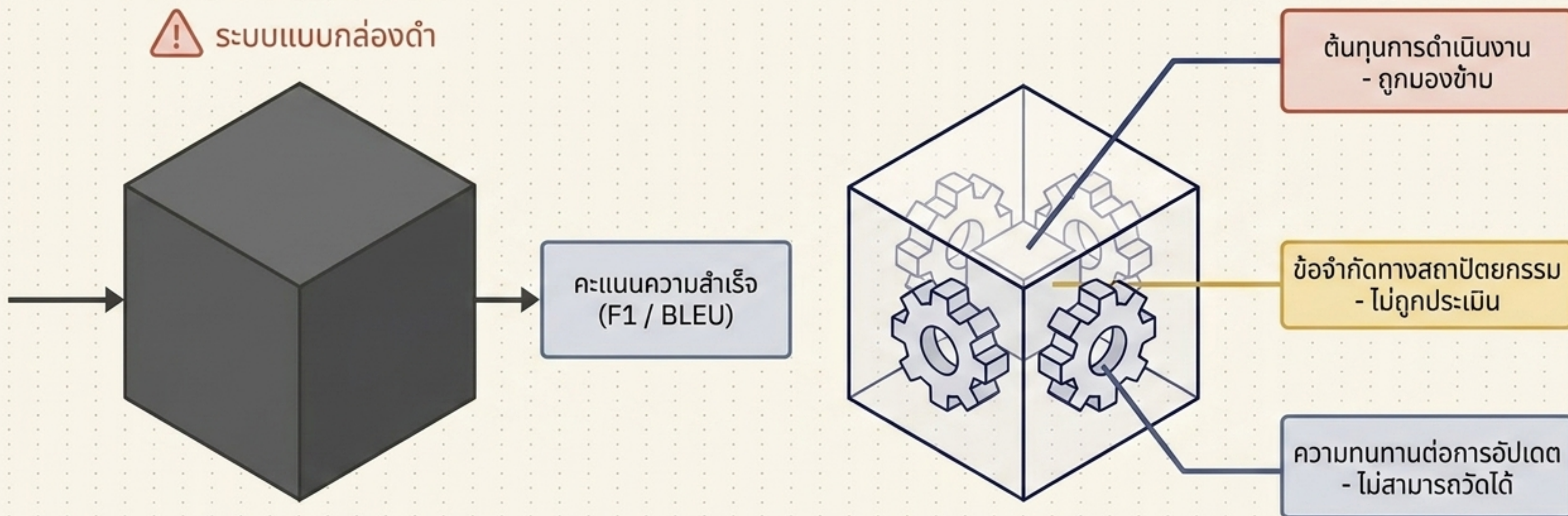


อดีต: RAG แบบพื้นฐาน



หน่วยความจำของ LLM Agent ไม่ใช่แค่คลังข้อมูลคงที่อีกต่อไป
แต่เป็นระบบจัดการข้อมูลที่ต้องเติบโตและปรับเปลี่ยนอยู่ตลอดเวลา

ปัญหาของระบบกล่องดำ: ทำไมตัวชี้วัดแบบเดิมถึงล่มเหลว?



การเปลี่ยนผ่านกระบวนการทัศนการประเมิน

แนวทางเดิม: มองระบบเป็นก้อนเดียว

มุมมอง

ระบบก้อนเดียว (Black Box)

ตัวชี้วัดหลัก

ความสำเร็จของงานปลายทาง (End-to-end task success เช่น F1, BLEU)

จุดอ่อน

เพิกเฉยต่อต้นทุนและกลไกการทำงานภายใน

แนวทางใหม่: การจัดการข้อมูลเต็มรูปแบบ

มุมมอง

โครงสร้างแบบโมดูลาร์ (Modular Architecture)

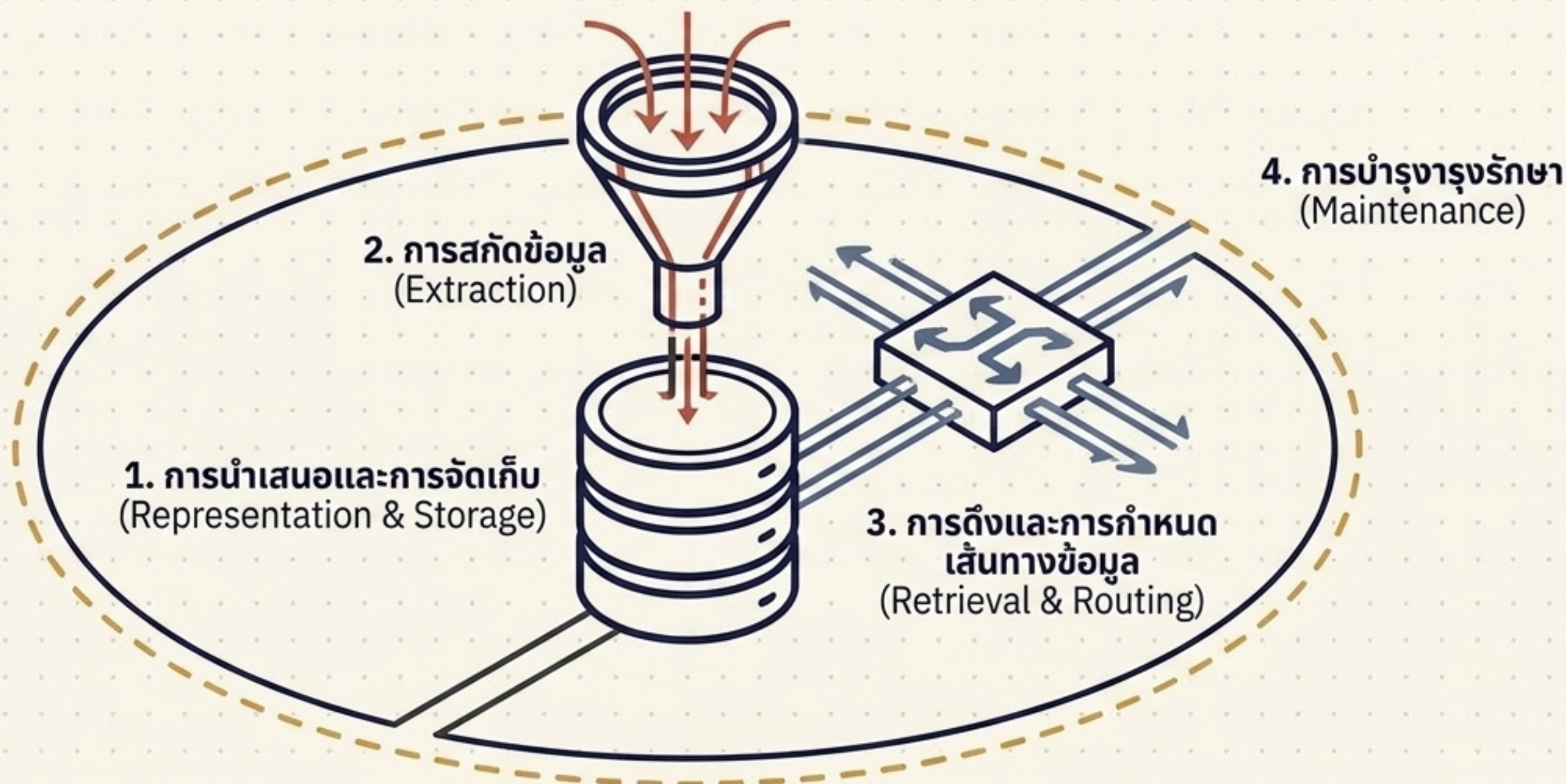
ตัวชี้วัดหลัก

ตัวชี้วัดระดับระบบ, ต้นทุน, และความเสถียร

จุดแข็ง

เปิดเผยให้เห็นถึงความท้าทายทางสถาปัตยกรรมที่แท้จริง

พิมพ์เขียวสถาปัตยกรรม: 4 โมดูลหลักของ Agent-Native Memory



รากฐานข้อมูล:
จัดเก็บโครงสร้างความรู้

วาล์วรับเข้า:
คัดกรองและสกัดข้อมูลสำคัญ

วาล์วส่งออก:
ดึงข้อมูลแม่นยำตามคำสั่ง

วงจรเบื้องหลัง:
ป้องกันการเสื่อมสภาพของระบบ

สเกลของการศึกษาที่ครอบคลุมที่สุด

Study Scale Dashboard

12

ระบบหน่วยความจำตัวแทน
(Representative Systems)

2

ระบบพื้นฐานสำหรับเปรียบเทียบ
(Reference Baselines)

5

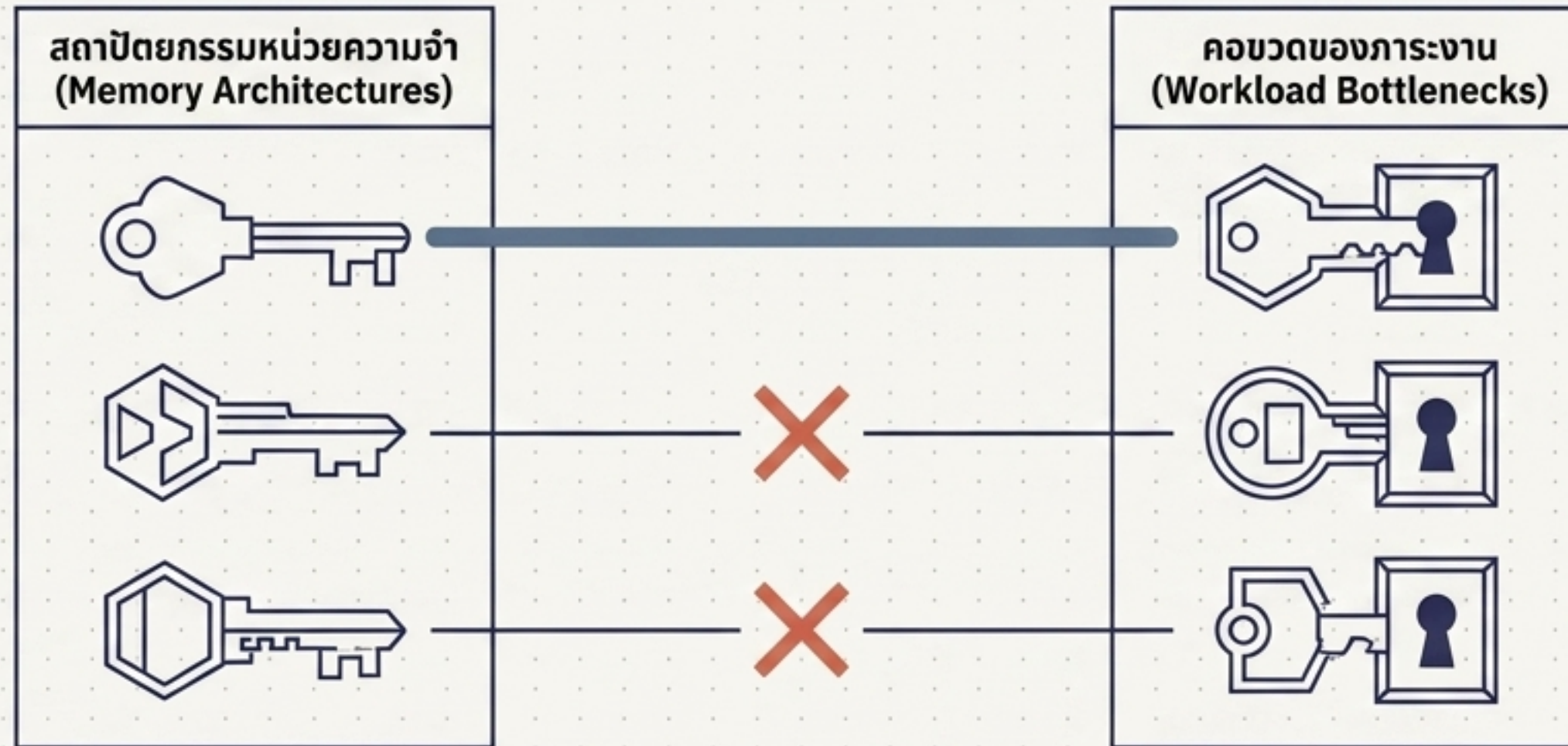
รูปแบบภาระงานหลัก
(Benchmark Workloads)

11

ชุดข้อมูลที่ใช้ทดสอบ
(Datasets)

การประเมินผลแบบ End-to-end ผ่านมุมมองการจัดการข้อมูล (Data Management)
เพื่อหาคำตอบว่าสถาปัตยกรรมใดทำงานได้ดีที่สุดในสถานการณ์จริง

ข้อมูลเชิงลึกที่ 1: จุดจบของสถาปัตยกรรมแบบครอบจักรวาล



ไม่มีสถาปัตยกรรมใดที่ชนะในทุกสถานการณ์
ประสิทธิภาพที่แท้จริงขึ้นอยู่กับว่า
โครงสร้างหน่วยความจำนั้นสอดคล้องกับคอขวดของภาระงานมากน้อยเพียงใด

ข้อมูลเชิงลึกที่ 2: จำและประสิทธิภาพผ่าน 4 มิติ

1. ความถูกต้องของการนำเสนอข้อมูล
(Representation Fidelity)



2. ความแม่นยำในการดึงข้อมูล
(Retrieval Precision)



3. ความถูกต้องในการอัปเดต
(Update Correctness)

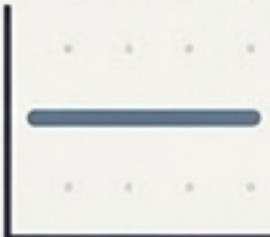


4. ความเสถียรในระยะยาว
(Long-horizon Stability)



การทดสอบแบบตัดออก (Ablation studies) พิสูจน์ว่า การออกแบบแต่ละโมดูลส่งผลกระทบต่อโดยตรงต่อมิติประสิทธิภาพเหล่านี้อย่างเฉพาะเจาะจง ซึ่งตัวชี้วัดแบบเดิมไม่สามารถจับสังเกตได้

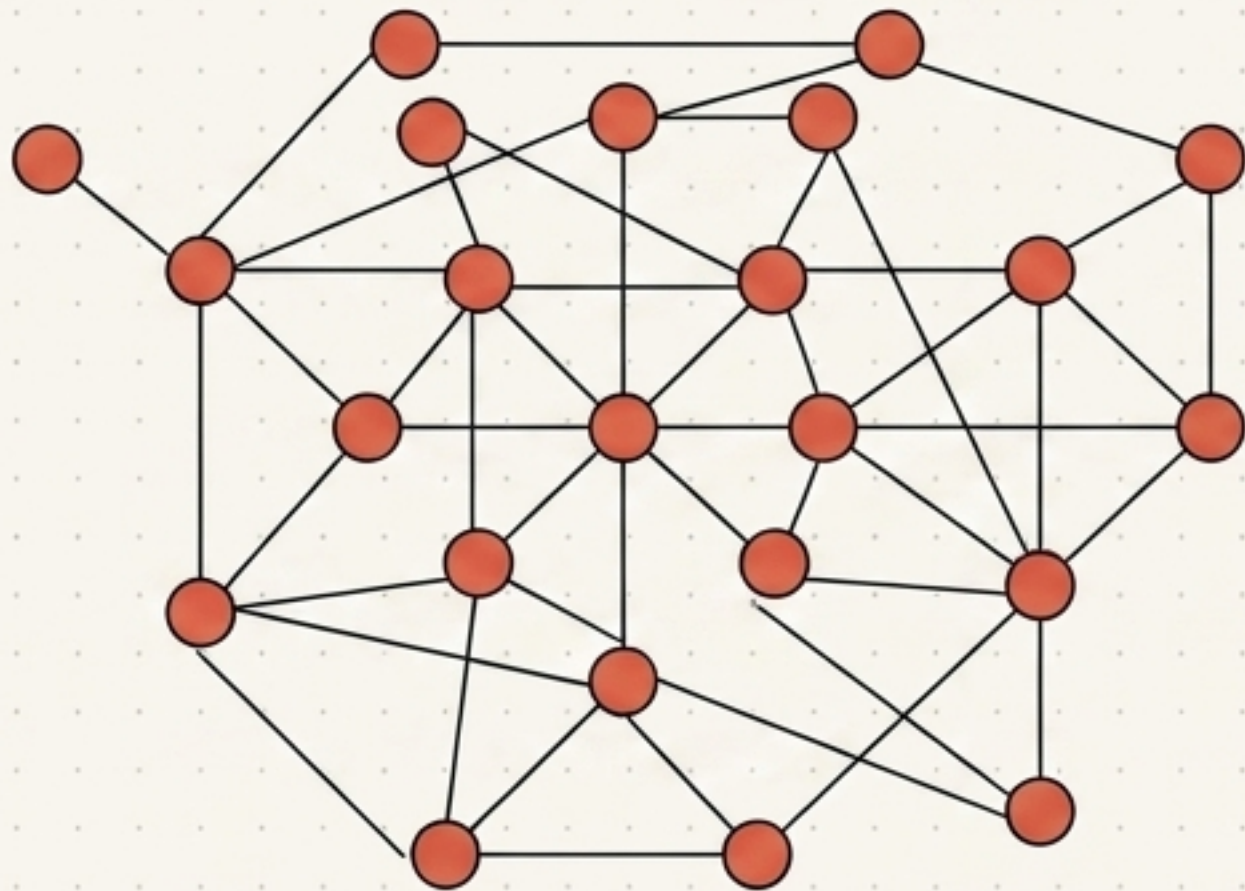
ข้อมูลเชิงลึกที่ 3: กลยุทธ์การบำรุงรักษา (Cost vs. Performance)

กลยุทธ์ (Strategy)	ต้นทุนการประมวลผล (Compute Cost)	ประสิทธิภาพเมื่อขยายขนาด (Efficiency at Scale)
การจัดระเบียบใหม่ทั่วทั้งระบบ (Global Reorganization)	สูงมหาศาล (Critical High) 	ลดลงอย่างรวดเร็ว (Degrades) 
การบำรุงรักษาเฉพาะจุด (Localized Maintenance)	ต่ำและควบคุมได้ (Low/Predictable) 	รักษาระดับได้ดีเยี่ยม (Sustained) 

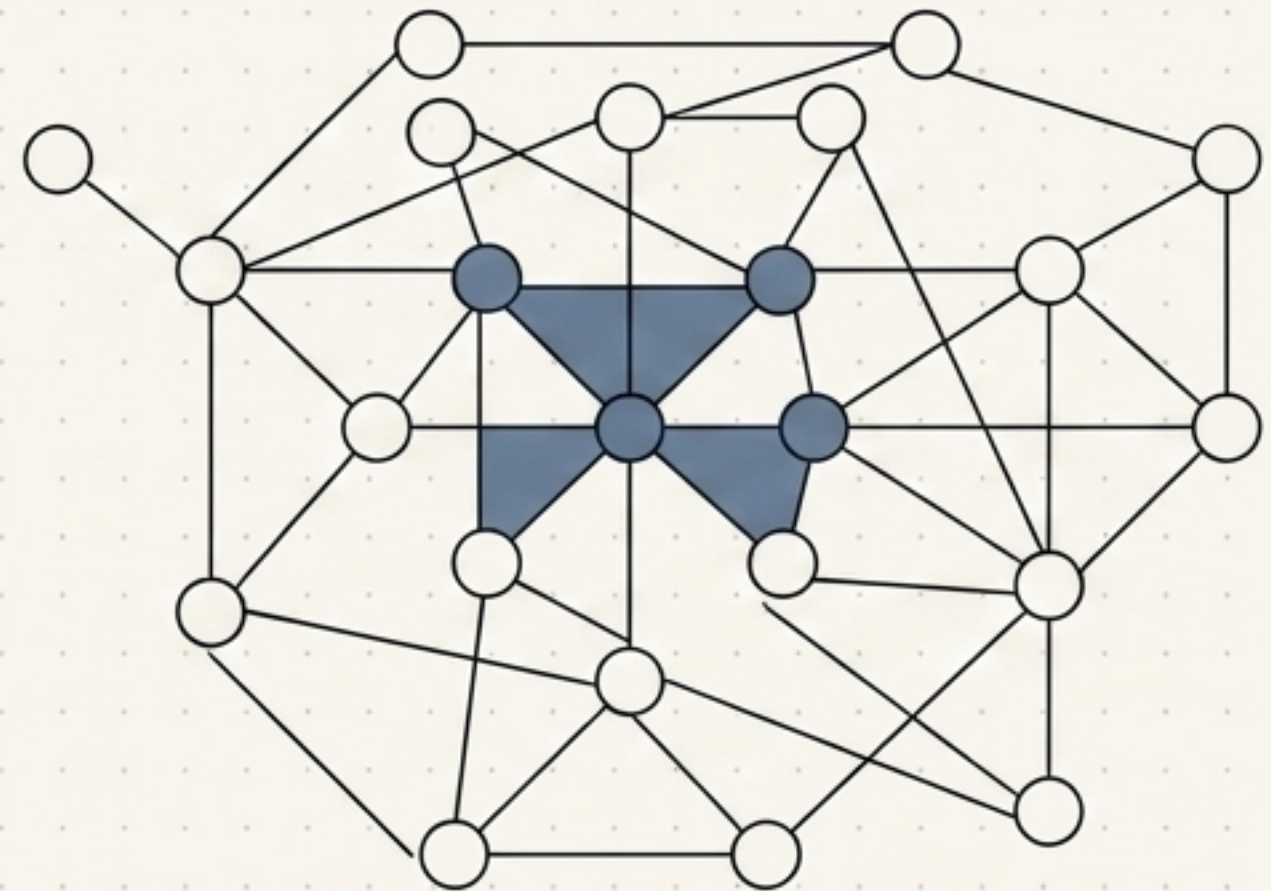
ภายใต้ภาระงานจริง Localized maintenance มีความคุ้มค่าด้านต้นทุนและประสิทธิภาพสูงอย่างมาก

เจาะลึก: ทำไมการบำรุงรักษาเฉพาะจุดถึงชนะเลิศ?

อัปเดตแบบทั่วทั้งระบบ (Global)



อัปเดตเฉพาะจุด (Localized)



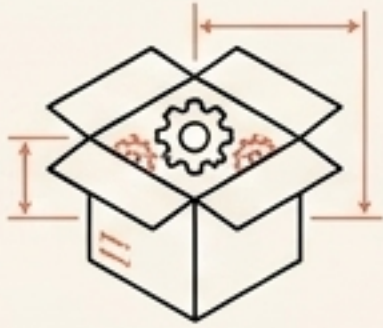
อัปเดตแบบศัลยกรรม (Surgical Updates) ช่วยลดการใช้พลังงานประมวลผลที่ไม่จำเป็นได้อย่างมหาศาล

บทสรุป: สมการสู่ความเป็น Agent-Native

$$\left[\begin{array}{l} \text{ความสำเร็จ} = \left(\frac{\text{สถาปัตยกรรมโมดูลาร์} \times \text{ความสอดคล้องกับคอบวดของงาน}}{\text{ต้นทุนการบำรุงรักษา}} \right) \\ \text{Success} = \left(\frac{\text{Modular Architecture} \times \text{Workload Alignment}}{\text{Maintenance Cost}} \right) \end{array} \right]$$

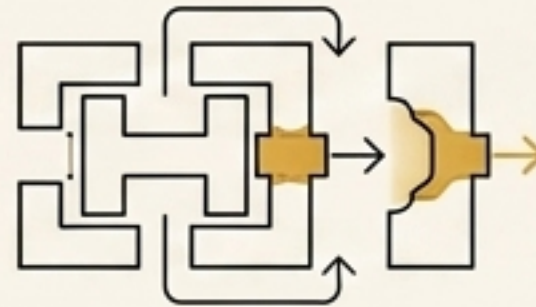
การสร้าง Agent ที่ดีไม่ได้อยู่ที่การเลือก Retriever ที่เก่งที่สุด แต่อยู่ที่การปรับแต่ง 4 โมดูลหลักให้ตรงกับลักษณะเฉพาะของงาน และใช้การบำรุงรักษาเฉพาะจุด (Localized) เพื่อควบคุมต้นทุนไม่ให้ออกนอกงบประมาณ

ก้าวต่อไปสู่สถาปัตยกรรม Agent-Native ที่แท้จริง



1. เลิกประเมินแบบกล่องดำ

ต้องออกแบบตัวชี้วัดที่สามารถวัดผลได้ลึกถึงระดับโมดูลภายใน ไม่ใช่แค่วัดผลปลายทาง



2. สถาปัตยกรรมต้องปรับเปลี่ยนได้

ไม่มีระบบใดดีที่สุดที่สุด
ต้องสามารถปรับโครงสร้าง
หน่วยความจำให้ตรงตาม
คอบขวดของข้อมูลในแต่ละงาน



3. ต้นทุนคือหัวใจสำคัญ

ระบบที่ยั่งยืนต้องจัดการ
วงจรชีวิตข้อมูลด้วย
Localized Maintenance
เพื่อไม่ให้ต้นทุนบานปลาย

ศึกษาข้อมูลวิจัยและ Source Code ฉบับเต็มได้ที่ GitHub: [OpenDataBox/MemoryData](#)